

Table of Contents

introducción	2
1. modelo entidad-relacion de base de datos.	2
a. grupo de tablas.....	2
b. descripcion de las tablas	3
i. ORGANIZATION.....	3
ii. USER	3
iii. PRODUCT.....	4
iv. SUPPLIER	4
v. BRANCH.....	5
vi. ESP32_NODE.....	5
vii. STORE_HOURS	6
viii. SHELF.....	6
ix. WEIGHT_READING.....	7
x. STOCK_EVENT	7
xi. ALERT	8
xii. ANOMALY_EVENT.....	8
xiii. INVENTORY_AUDIT	9
xiv. AUDIT_LINE	9
xv. PURCHASE_ORDER.....	10
xvi. SUPPLIER_RECEIPT	10
xvii. BUSINESS_REPORT.....	11
xviii. SEASONAL_PATTERNS.....	11
xix. ORGANIZATION_TOKEN.....	12
xx. BRANCH_INVITATION	12
c. relaciones principales.....	13
a. visión general de la arquitectura.....	14
b. descripcion de las capas y componentes	14
i. capa fisica:.....	14
ii. edge gateway	14
iii. cloud compute - raulway.app	15
iv. backend as a service - Supabase	15
v. servicios externos	16

c.	Flujo de datos	16
d.	Stack tecnológico.....	17
3.	case - hardware	19
a.	case principal	19
b.	case secundario	20
c.	estante con celda	21
4.	MOCKUP	22
5.	cambios y justificaciones.....	23
a.	cambio en los case.....	23

introducción

RackIQ es un sistema IoT de gestion de inventario diseñado para **PyMEs** utiliza celdas de carga instaladas en los estantes fisicos para detectar cambios de peso en tiempo real, inferir eventos de venta y mantener un inventario actualizado sin intervencion humana.

se presentara la arquitectura que va conformar el proyecto desde el analisis de la base de datos como las tecnologias a utilizar y case del equipo fisico y el porque ese diseno

1. modelo entidad-relacion de base de datos.

a. grupo de tablas

las tablas las podremos organizar en 5 grupos

Grupo	tablas	responsabilidad
organizacion y usuarios	ORGANIZATION, USER, BRANCH, ORGANIZATION_TOKEN, BRANCH_INVITATION	Multi-tenancy, autenticación, roles y control de acceso
Hardware e infraestructura	ESP32_NODE, STORE_HOURS	Nodos físicos, heartbeat, horarios de tienda

Catálogo y configuración	PRODUCT, SUPPLIER, SHELF	Productos, proveedores, estantes y umbrales
Datos de sensores y eventos	WEIGHT_READING, STOCK_EVENT, ALERT, ANOMALY_EVENT	Flujo de datos en tiempo real desde la Raspberry Pi
Lógica de negocio	INVENTORY_AUDIT, AUDIT_LINE, PURCHASE_ORDER, SUPPLIER_RECEIPT, BUSINESS_REPORT, SEASONAL_PATTERN	Auditorías, pedidos, reportes y patrones de ML

b. descripción de las tablas

i. ORGANIZATION

campo	tipo	clave	descripcion
id	uuid	PK	Identificador único de la organización
name	string		Nombre del negocio
slug	string		identificador URL amigable unico
plan_type	string		plan de suscripcion: free - pro
cerared_at	timestamp		fecha de registro de la organizacion

ii. USER

campo	tipo	clave	descripción
id	uuid	PK	Identificador único del usuario (sincronizado con Supabase Auth)
organization_id	uuid	FK	Organización a la que pertenece
email	string		correo - usado para login y alertas
name	string		nombre completo del usuario
role	string		rol: owner, manager, admin, staff
push_token	string		Token FCM del dispositivo móvil para notificaciones push U07
created_at	timestamp		fecha de alta del usuario

iii. PRODUCT

Catálogo de productos de la organización. El campo `unit_weight_grams`, permite al sistema convertir un delta de peso (gramos) en unidades vendidas. Un estante solo puede tener asignado un tipo de producto a la vez.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del producto
organization_id	uuid	FK	Organización dueña del producto
name	string		Nombre del producto (ej. Leche Lala 1L)
sku	string		Código de identificación del producto
unit_weight_grams	float		Peso por unidad en gramos — base para calcular unidades por peso
cost_price	float		Precio de costo para cálculo de rentabilidad U09
sale_price	float		Precio de venta al público
category	string		Categoría del producto (ej. lácteos, botanas)
created_at	timestamp		Fecha de registro

iv. SUPPLIER

Proveedores registrados por la organización. El campo `preferred_channel` determina por qué medio el Worker notifica al dueño cuando hay stock bajo en modo manual: WhatsApp, email o Telegram. El `order_template` contiene el mensaje personalizado de pedido.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del proveedor
organization_id	uuid	FK	Organización a la que pertenece
name	string		Nombre del proveedor
contact_email	string		Email de contacto para alertas de pedido F06
whatsapp_number	string		Número de WhatsApp para notificaciones
preferred_channel	string		Canal preferido: whatsapp / email / telegram

order_template	string		Plantilla de mensaje de pedido personalizable
created_at	timestamp		Fecha de registro

v. **BRANCH**

el timezone es importante, para el registro, detecta retiros fuera del horario de tienda, como posibles anomalías. una organización puede tener múltiples sucursales

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único de la sucursal
organization_id	uuid	FK	Organización a la que pertenece
name	string		Nombre de la sucursal (ej. Sucursal Norte)
address	string		Dirección física
timezone	string		Zona horaria para detección de actividad fuera de horario F07.1
is_active	boolean		Indica si la sucursal está operativa
created_at	timestamp		Fecha de alta

vi. **ESP32_NODE**

Registro de cada microcontrolador ESP32 instalado en una sucursal. El sistema actualiza last_heartbeat_at cada vez que recibe el ping periódico del nodo. Si el tiempo transcurrido supera el umbral configurado, is_online se marca false y se dispara una alerta de tipo offline_node

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del nodo ESP32
branch_id	uuid	FK	Sucursal donde está instalado
mac_address	string		Dirección MAC — identifica el dispositivo físico en la red
firmware_version	string		Versión del firmware C/ESP-IDF instalado
last_heartbeat_at	timestamp		Último ping recibido — detecta nodos caídos
is_online	boolean		Estado actual de conexión del nodo
wifi_rssi	float		Potencia de señal WiFi — diagnóstico de conectividad

vii. STORE_HOURS

Horarios de operación de cada sucursal por día de la semana. El Edge Service consulta estos registros para determinar si una variación de peso ocurre fuera del horario normal, condicion necesario para detectar posibles robos fuera de horario

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador
branch_id	uuid	FK	Sucursal a la que aplica el horario
day_of_week	int		Día de la semana (0=lunes ... 6=domingo)
open_time	time		Hora de apertura
close_time	time		Hora de cierre — referencia para detectar retiros fuera de horario F07.1

viii. SHELF

Tabla central del modelo operativo. Vincula el hardware (ESP32, canal HX711) con el producto asignado y la sucursal. Almacena la configuración de calibración, los umbrales de stock bajo y el modo de respuesta ante stock bajo.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del estante
branch_id	uuid	FK	Sucursal donde está ubicado
product_id	uuid	FK	Producto asignado — un solo tipo por estante U04
esp32_node_id	uuid	FK	Nodo ESP32 que lo monitorea
name	string		Nombre descriptivo (ej. Estante A3)
hx711_channel	int		Canal del HX711 (0–4) — soporta hasta 5 estantes por ESP32 F02
tare_weight_grams	float		Peso del estante vacío — tara F11
scale_factor	float		Factor de calibración del HX711 F11
max_capacity_grams	float		Capacidad máxima del estante en gramos
low_stock_thresh_hold_pct	float		Umbral de stock bajo en porcentaje F05

low_stock_thres hold_kg	float		Umbral de stock bajo en kilogramos F05
alert_mode	string		Modo de alerta: auto (auto-pedido) / manual (notificación) F06
last_calibrated_ at	timestamp		Última vez que se calibró desde la app F11
is_active	boolean		Indica si el estante está en operación

ix. WEIGHT_READING

Registro de cada lectura de peso recibida desde la Raspberry Pi. Los registros de esta tabla son la fuente primaria que se ingesta también en InfluxDB Cloud para análisis de series de tiempo. raw_weight_grams es el valor bruto del sensor; net_weight_grams ya tiene descontada la tara configurada en SHELF.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único de la lectura
shelf_id	uuid	FK	Estante que generó la lectura
raw_weight_grams	float		Peso bruto reportado por el HX711 antes de filtrado
net_weight_grams	float		Peso neto después de restar la tara F11
recorded_at	timestamp		Timestamp preciso de la lectura — almacenado en InfluxDB para series de tiempo

x. STOCK_EVENT

Eventos consolidados de cambio de inventario. A diferencia de WEIGHT_READING (que registra cada lectura), STOCK_EVENT solo se crea cuando el Edge Service confirma un retiro definitivo (is_consolidated = true) tras el período de debounce. units_delta se calcula dividiendo weight_delta_grams entre unit_weight_grams del producto.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del evento
shelf_id	uuid	FK	Estante donde ocurrió
event_type	string		Tipo: withdrawal (retiro) / restock (surtido) / adjustment (ajuste)
weight_delta_grams	float		Cambio de peso en gramos (negativo = retiro)

units_delta	float		Unidades equivalentes calculadas con unit_weight_grams
weight_before	float		Peso del estante antes del evento
weight_after	float		Peso del estante después del evento
occurred_at	timestamp		Momento del evento
is_consolidated	boolean		true = retiro definitivo confirmado

xi. ALERT

Alertas activas del sistema. Cubre todos los tipos de alerta: stock bajo, anomalía de robo, nodo desconectado, discrepancia de proveedor y peso desconocido en estante. El campo metadata en JSONB almacena datos específicos del tipo de alerta sin requerir columnas adicionales.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único de la alerta
shelf_id	uuid	FK	Estante que la originó
branch_id	uuid	FK	Sucursal afectada
alert_type	string		Tipo: low_stock / anomaly / offline_node / supplier_discrepancy / unknown_weight
status	string		Estado: active / resolved / dismissed
metadata	jsonb		Datos adicionales según el tipo
triggered_at	timestamp		Momento en que se disparó la alerta
resolved_at	timestamp		Momento en que fue resuelta
resolved_by	uuid	FK	Usuario que resolvió la alerta

xii. ANOMALY_EVENT

Detalle de eventos anómalos detectados por el sistema. Se distinguen dos tipos: excessive_rate (velocidad de retiro inusualmente alta) y out_of_hours (retiro fuera del horario registrado en STORE_HOURS). La trazabilidad temporal de estos eventos permite al dueño revisar las cámaras de seguridad con timestamps exactos.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único
shelf_id	uuid	FK	Estante donde se detectó
anomaly_type	string		Tipo: excessive_rate (velocidad) / out_of_hours (fuera de horario)

weight_delta_grams	float		Volumen de peso involucrado
rate_per_minute	float		Velocidad de retiro en gramos por minuto
out_of_hours	boolean		true si ocurrió fuera del horario de tienda
status	string		Estado: pending / confirmed / dismissed
detected_at	timestamp		Momento de detección

xiii. INVENTORY_AUDIT

Corte de inventario por sucursal y período. Solo usuarios con rol manager u owner pueden crear y cerrar auditorías. Cada auditoría contiene múltiples líneas en AUDIT_LINE, una por estante auditado.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único de la auditoría
branch_id	uuid	FK	Sucursal auditada
created_by	uuid	FK	Usuario que abrió la auditoría (requiere rol admin/manager) U08
period_start	timestamp		Inicio del período auditado
period_end	timestamp		Fin del período auditado
status	string		Estado: open / closed
closed_at	timestamp		Momento de cierre del corte

xiv. AUDIT_LINE

Línea detallada de la auditoría por estante. Compara las unidades que el sensor reportó como retiradas (sensor_units_out) contra las unidades cobradas en caja (pos_units_sold). La diferencia (discrepancy_units) es clasificada manualmente por el administrador según su naturaleza.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único de la línea
audit_id	uuid	FK	Auditoría a la que pertenece
shelf_id	uuid	FK	Estante auditado
sensor_units_out	float		Unidades retiradas según el sensor

t			
pos_units_sold	float		Unidades cobradas en caja (ingreso manual)
discrepancy_units	float		Diferencia: sensor_units_out - pos_units_sold
resolution_type	string		Clasificación: confirmed_loss / expired / cashier_error / none
resolved_by	uuid	FK	Usuario que clasificó la discrepancia
resolved_at	timestamp		Momento de resolución

xv. PURCHASE_ORDER

Pedidos generados al detectar stock bajo. En modo automático, el Worker crea el registro y notifica al proveedor por el canal configurado. En modo manual, solo crea el registro y notifica al dueño para que él decida.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del pedido
shelf_id	uuid	FK	Estante que originó el pedido por stock bajo
supplier_id	uuid	FK	Proveedor al que va dirigido
trigger_type	string		Origen: automatic (auto-stock) / manual (iniciado por el dueño) F06
quantity_kg	float		Cantidad solicitada en kilogramos
status	string		Estado: pending / sent / confirmed / cancelled
created_at	timestamp		Momento de creación
sent_at	timestamp		Momento en que se envió la notificación al proveedor

xvi. SUPPLIER_RECEIPT

Registro del Modo Surtido. Cuando llega un proveedor, el empleado activa este modo en la app. El sistema mide el peso total añadido al estante y lo compara contra lo declarado. Si discrepancy_grams supera el umbral, se genera una alerta de tipo supplier_discrepancy antes de que el repartidor se retire.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del recibo
shelf_id	uuid	FK	Estante donde se recibió la mercancía

created_by	uuid	FK	Empleado que activó el Modo Surtido
declared_weight_grams	float		Peso declarado en la factura o por el repartidor
measured_weight_grams	float		Peso real medido por el sensor
discrepancy_grams	float		Diferencia entre lo declarado y lo medido
status	string		Estado: ok / discrepancy_reported
received_at	timestamp		Momento de recepción

xvii. BUSINESS_REPORT

Metadatos de los reportes periódicos generados por el Worker Celery. El campo payload JSONB contiene el contenido completo del reporte: KPIs del período, top de ventas, alertas acumuladas y sugerencias generadas por el LLM. El archivo físico (PDF o Excel) se almacena en Supabase Storage

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del reporte
branch_id	uuid	FK	Sucursal del reporte
report_type	string		Tipo: weekly / monthly / quarterly / annual
period_start	timestamp		Inicio del período cubierto
period_end	timestamp		Fin del período cubierto
payload	jsonb		Contenido del reporte: KPIs, top ventas, alertas acumuladas, sugerencias IA
format	string		Formato generado: pdf / excel
status	string		Estado: generating / ready / failed
generated_at	timestamp		Momento en que el Worker terminó de generarlo

xviii. SEASONAL_PATTERNS

Patrones de venta aprendidos por el Worker ML. Se actualiza periódicamente con base en el historial de STOCK_EVENT e InfluxDB. avg_sales_units indica el promedio histórico de ventas para ese producto en esa semana y día específicos, permitiendo al sistema sugerir cantidades de pedido y detectar sobre-stock estacional.

Campo	Tipo	Clave	Descripción
-------	------	-------	-------------

id	uuid	PK	Identificador único
product_id	uuid	FK	Producto al que aplica el patrón
branch_id	uuid	FK	Sucursal específica
week_of_year	int		Semana del año (1–52)
day_of_week	int		Día de la semana (0–6)
avg_sales_units	float		Promedio de unidades vendidas en ese período F08
confidence_score	float		Confianza del modelo (0.0–1.0) según historial disponible
updated_at	timestamp		Última actualización del patrón por el Worker ML

xix. ORGANIZATION_TOKEN

Tokens de API para autenticar la Raspberry Pi hacia el backend FastAPI. Cada sucursal tiene su propio token. La API verifica este token en cada petición de sincronización de datos (POST /readings). El campo last_used_at permite detectar tokens inactivos o comprometidos

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del token
organization_id	uuid	FK	Organización propietaria
token	string		Token de API para autenticación de la Raspberry Pi hacia el backend
label	string		Etiqueta descriptiva (ej. RPi Sucursal Norte)
is_active	boolean		Indica si el token está habilitado
created_by	uuid	FK	Usuario que generó el token
created_at	timestamp		Fecha de creación
last_used_at	timestamp		Último uso — ayuda a detectar tokens inactivos

xx. BRANCH_INVITATION

Sistema de invitaciones para añadir usuarios a una organización. El dueño ingresa el email del nuevo empleado, el sistema genera un token único que se envía por email. Al hacer clic, el invitado crea su cuenta ya vinculada a la organización correcta.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único
organization_id	uuid	FK	Organización que hace la invitación
invited_by	uuid	FK	Usuario que envió la invitación
token	string		Token único enviado por email al invitado
email	string		Correo del invitado
status	string		Estado: pending / accepted / expired
expires_at	timestamp		Fecha de expiración del token
used_at	timestamp		Momento en que el invitado aceptó

c. relaciones principales

Tabla origen	Cardinalidad	Tabla destino	Significado
ORGANIZATION	1 → N	BRANCH	Una organización tiene varias sucursales
ORGANIZATION	1 → N	USER	Una organización tiene varios usuarios con diferentes roles
BRANCH	1 → N	SHELF	Una sucursal tiene múltiples estantes
SHELF	N → 1	PRODUCT	Cada estante tiene asignado un solo producto (U04)
SHELF	N → 1	ESP32_NODE	Hasta 5 estantes comparten un nodo ESP32 (F02)
WEIGHT_READING	N → 1	SHELF	Cada lectura de sensor pertenece a un estante
STOCK_EVENT	N → 1	SHELF	Cada evento de inventario ocurre en un estante
ALERT	N → 1	SHELF + BRANCH	Una alerta referencia el estante y su sucursal
INVENTORY_AUDIT	1 → N	AUDIT_LINE	Una auditoría contiene una línea por estante
PURCHASE_ORDER	N → 1	SUPPLIER	Cada pedido va a un proveedor específico
SEASONAL_PATTERN	N → 1	PRODUCT + BRANCH	Patrón específico por producto y sucursal

2. diagrama de arquitectura de software

a. visión general de la arquitectura

la arquitectura de software esta organizada en cinco capas separadas por sus responsabilidades

- i. **capa fisica:** hardware de sensores instalado en cada sucursal
- ii. **edge gateway:** procesamiento local en la raspberry pi
- iii. **cloud computing:** logica de negocio
- iv. **backend as service:** base de datos, autenticacion y storage
- v. **clientes finales:** aplicacion movil con **react-native** para **iOS y Android**

b. descripcion de las capas y componentes

i. capa fisica:

la capa fisica puede constar de **2 nodos** pero como un inicio tiene que tener el **nodo principal**. este nodo principal constara de los siguientes componentes

componente	especificacion	funcion en RacIQ
celda de carga x 5	sensor analogico	mide el peso de los productos en cada estante
HX711 x 5	ADC de 24 bits	amplifica y digitaliza la senal
ESP32	controlador para los sensores HX711	transmite datos cuando encuentra un cambio real que supera el umbral configurado.
Raspberry pi	concentrador de datos local.	recibe los datos transmitidos por los ESP32, empaqueta los datos y transmite al servidor evitando saturar la red WIFI

ii. edge gateway

corre 3 contenedores **Docker** con responsabilidades separadas

contenedor	tecnologia	responsabilidad
docker 1	Mosquitto : 1883	broker MQTT local. recibe las publicaciones del/los ESP32 por WIFI
docker 1	python : FashAPI	suscribe al broker, filtra ruido, aplica debounce consolidado, implementa store & forward.
docker 3	SQLite	buffer offline: almacena lecturas pendientes cuando no hay internet, se sincroniza progresivamente al reconectar.

la raspberry pi no toma decisiones del negocio, su responsabilidad es tomar los datos enviados por el/los esp32, limpiarlos localmente y enviarlos de forma confiable a la nube garantizando utilizar poco ancho de banda y operación continua, aunque el internet falle.

iii. cloud compute - raulway.app

el hosting en donde corra el código de backend en python.
aloja 2 contenedores mas **redis** como broker interno.

contenedor	tecnologia	responsabilidad
docker 4	python: FastAPI	REST API principal. Recibe datos de la RPi, gestiona inventario, valida JWT de Supabase Auth, expone endpoints a la app móvil, publica eventos por WebSocket
docker 5	python : Celery	Worker de tareas en segundo plano. Genera reportes PDF/Excel, ejecuta ML de estacionalidad, llama al LLM para sugerencias, despacha alertas
docker 5	redis : Upstash	Broker interno de Celery. Cola de tareas asíncronas, caché de sesiones JWT y pub/sub para WebSockets en tiempo real

la separación entre la API (**Docker 4**) y el worker (**Docker 5**) evita que realizar tareas pesadas como el generar un PDF o llamar al LLM la ejecute directamente, esto puede bloquear la aplicación, en este caso encola la tarea en **Redis** y la API sigue respondiendo a peticiones. el **Worker** la procesa en paralelo sin bloquear

iv. backend as a service - Supabase

Supabase proporciona los servicios de persistencia y comunicación en tiempo real. **PostgreSQL** con servicios integrados, conectados desde **FastAPI**

servicio	usdo en RackIQ
PostgreSQL	Base de datos relacional principal. Almacena inventario, usuarios, sucursales, umbrales, pedidos, auditorías y configuración del sistema
Supabase Auth	Autenticación de usuarios con JWT, OAuth y gestión de roles (dueño, empleado, administrador). Elimina la necesidad de construir el sistema de login desde cero
Supabase Storage	Almacenamiento de archivos generados por el Worker:

	reportes PDF, Excel y assets estáticos.
Supabase Realtime	Canal de WebSockets que permite suscribirse a cambios en PostgreSQL. La app móvil recibe alertas de stock bajo en tiempo real sin polling, alerta sobre desconexión

v. servicios externos

servicio	plataforma	uso en RackIIQ
push notifications	firebase FCM	alertas críticas al dispositivo de los usuarios aunque esté cerrada la app.
email report	Resend.com	envió automático de reportes PDF y excel adjuntos.
LLM - sugerencias IA	OpenIA / DeepSeek / Gemini / Claude	genera sugerencias para inventario estancado, aprendizaje del negocio.
contacto manual con proveedor	whatsapp / telegram / email	el usuario que requiera realizar pedido manualmente podría realizar la petición desde la app e enviarlo por algún medio de contacto.
series de tiempo	influxDB Cloud	Almacena el historial completo de lecturas de peso (weight_readings) para análisis de patrones, aprendizaje estacional y generación de gráficas

C. Flujo de datos

#	componente	acción
1	ESP32 -> mosquito	el ESP32 detecta un cambio de peso superior al umbral y publica un mensaje MQTT al broker local por WIFI
2	mosquito -> edge service	el edge service suscrito al broker recibe el dato, aplica filtro de ruido y debounce. si no hay internet lo guarda en SQLite
3	edge service -> API FastAPI	El dato limpio se envía por HTTPS a la API en Railway. Si la conexión regresó, se envían los datos acumulados en SQLite de forma progresiva
4	API → Supabase + InfluxDB	La API guarda el evento en PostgreSQL (Supabase) para el inventario y en InfluxDB para el historial de series de tiempo
5	API → Redis → Worker (Celery)	La API encola una tarea en Redis para que el Worker evalúe si se activó algún umbral de stock bajo o anomalía
6	Worker → FCM / Notificación	Si se detectó stock bajo o anomalía, el Worker despacha una notificación push via FCM y una alerta por el canal configurado (WhatsApp/email)

7	Supabase Realtime → App	La app móvil recibe la alerta en tiempo real por WebSocket sin necesidad de hacer polling al servidor
---	-------------------------	---

d. Stack tecnológico

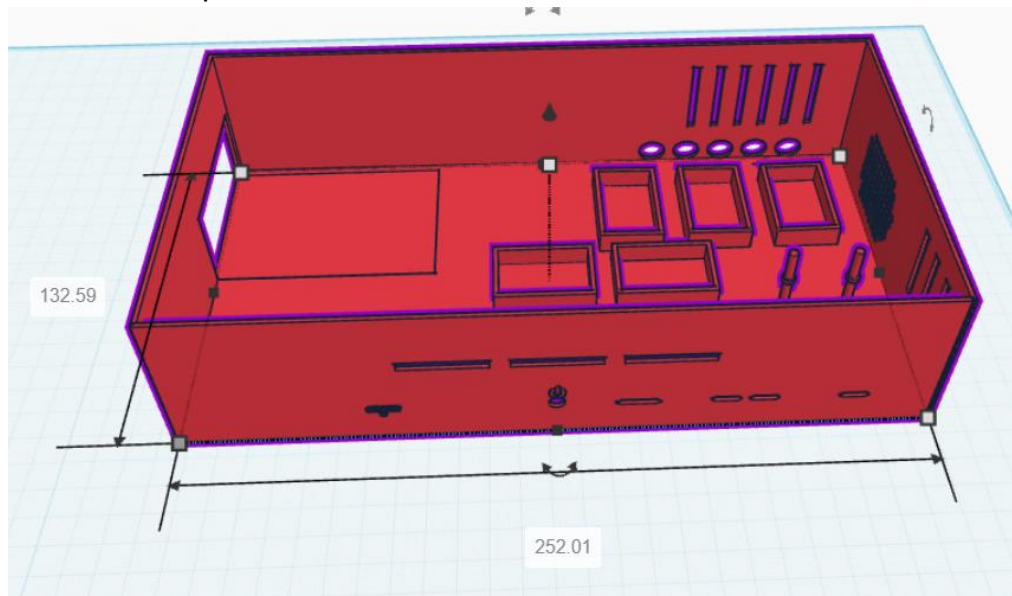
servicio / rol	plataforma	tecnologia	tipo de cobro	free tier	costo / mes (dóls)
Compute — API REST (Docker 4)	Railway.app	Python · FastAPI	por uso	\$5 créditos / mes	~\$5 -15
Compute — Worker Celery (Docker 5)	Railway.app	Python · Celery	por uso	\$5 créditos / mes	~\$5 -15
Cola de tareas — Redis (Docker 6)	Upstash serverless	Redis	por uso	10k comandos / día - gratis	\$0.2 por 100k comandos (~\$1 - 5)
Base de datos relacional	Supabase free tier	PostgreSQL 16	mensual fijo	500 MB plan free por siempre	\$25 - plan pro
Autenticación de usuarios	Supabase Auth	JWT · OAuth · RLS	por uso	50,000 MAU gratis	\$0.00325 por MAU extra (incluido en plan pro)
Storage archivos (PDF/Excel)	Supabase Storage	S3-compatible API	por uso	1 GB gratis	\$0.021/GB extra (incluido en plan pro)
WebSockets — alertas realtime	Supabase Realtime	WebSockets sobre PG	mensual fijo	200 conexiones simultaneas	incluido en plan pro
Series de tiempo — sensores	InfluxDB Cloud	InfluxDB	mensual fijo	plan gratis con limite writes	\$0 gratis
Push notifications	Firebase FCM	@react-native-firebase	free	iliminado	\$0 gratis
Email con reportes adjuntos	Resend.com	API HTTP	mensual fijo	3,000 email/ mes	\$20 - 50k emails
alertas proveedor	whatsapp / email / telegram API	REST API - python SDK	por uso	1,000 conversaciones	~\$0.05 - 0.08 por conversacion
Sugerencias IA inventario	OpenAI / DeepSeek / Gemini / claude	LLM API (Python SDK)	por uso	pay-per-use	pay-per-use
App móvil (iOS + Android)	Expo · EAS Build	React Native	free	free	\$0 gratis
TOTOAL MENSUAL ESTIMADO					utilizando free tiers ~\$10
					si se supera el plan gratis o se requiere mas potencia costo subira ~\$60 - 120 / mes

3. case - hardware

el sistema se compone de al menos 2 case. **1 case principal, 1 case con el estante y si es necesario un case secundario.**

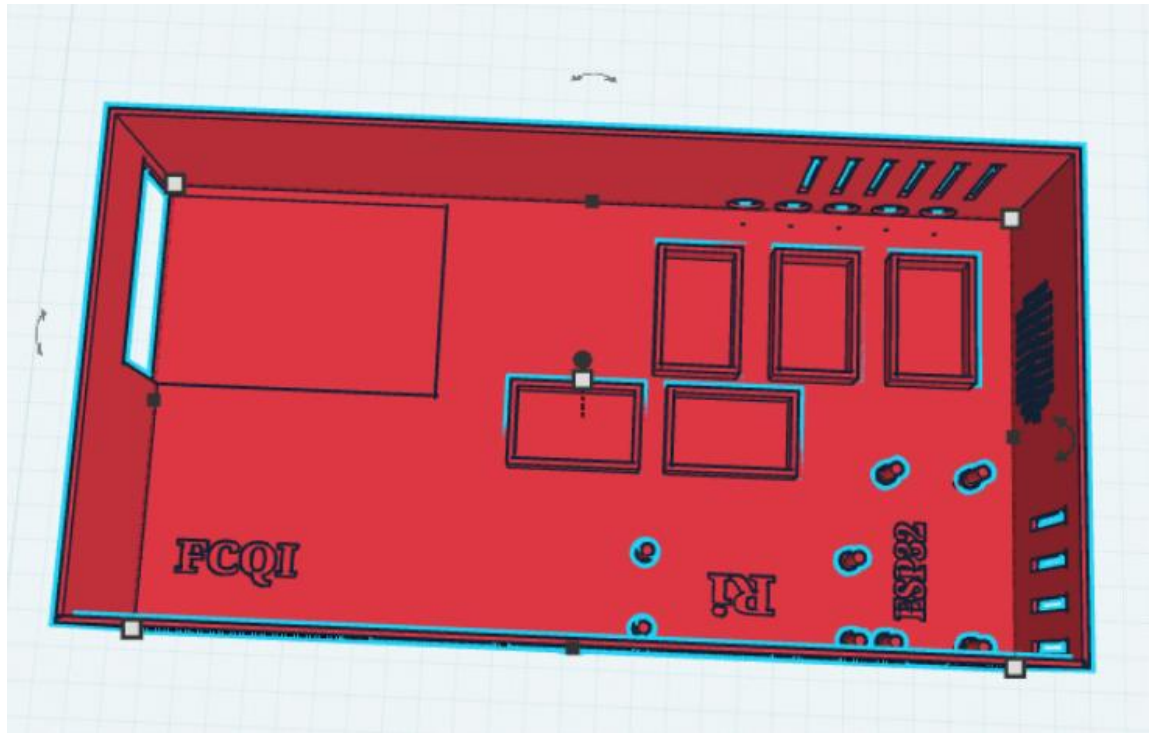
a. case principal

este sera el nodo principal, toda sucursal que adquiriera el producto de **RackIQ** debería de tenerlo, constara de **1 ESP32, 1 raspberry pi zero 2 w, 5 modulos HX711 y sus 5 celdas de carga.** pueden utilizarse todas o no, a petición del cliente.



tendra salida por donde pasaran los cables blindados hacia las celdas de carga, importante que esten blindados para evitar interferencias con la senal que manda las celdas

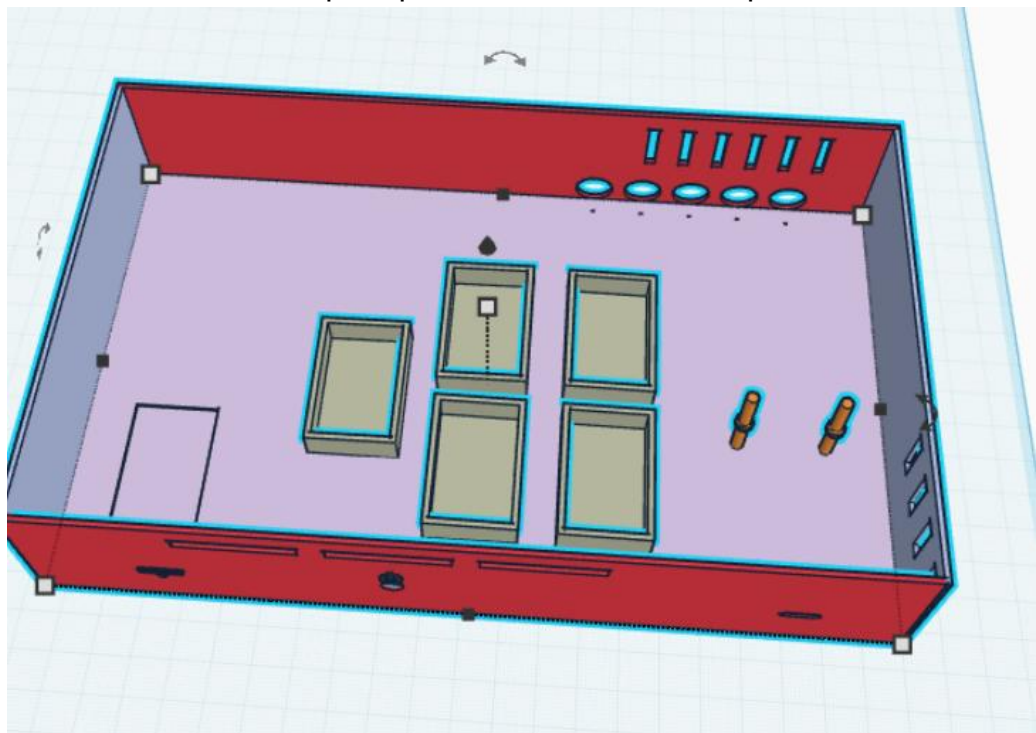
los estantes pueden estar hasta **una distancia maxima de 5 metros del case donde se encuentran los modulos HX711**, tendra una **fuelle conmutada de 5v a 10 A.** para alimentar todo lo necesario del nodo principal



b. case secundario

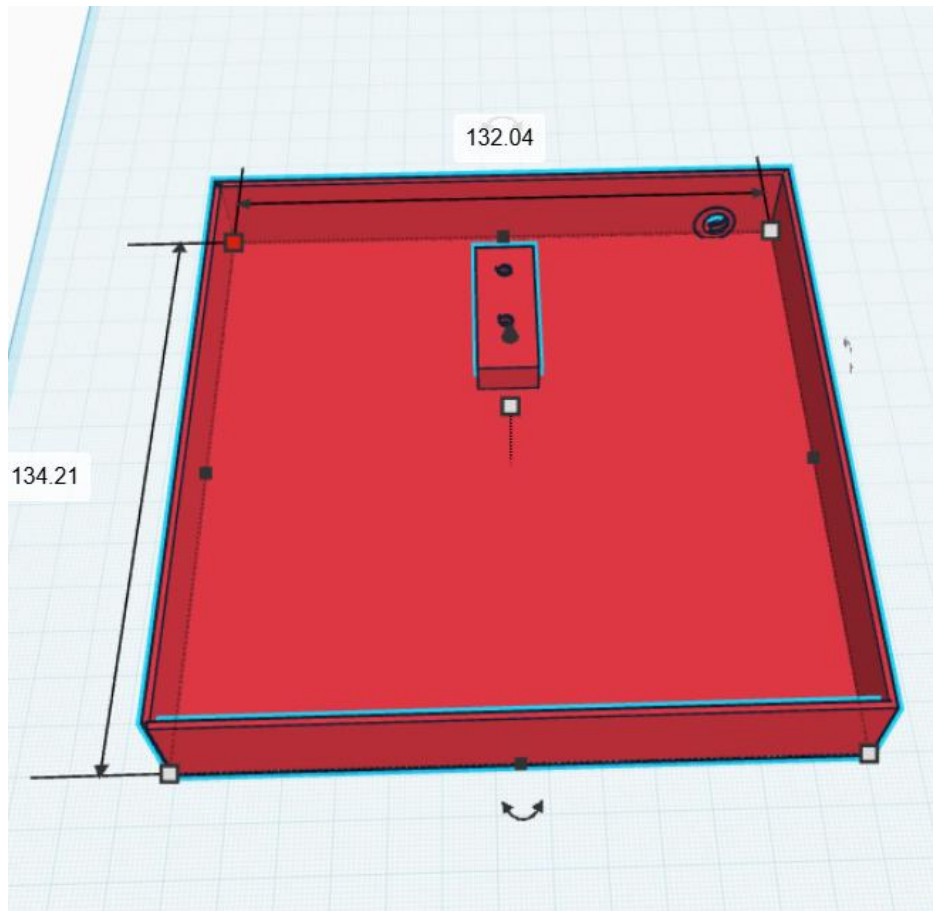
este nodo es muy parecido al nodo pero este solo constara de **1 ESP32** con la capacidad de tener **5 modulos HX711** con **sus celdas de carga respectivamente.**

solo en caso de que se necesiten mas. este constara de un convertidor de **AC-DC de 5V - 1A** para poder alimtnar a los dispositivos.



c. estante con celda

el ultimo case necesario es donde estarán las celdas de carga, un estante diseñado a medida con capacidad de cargar hasta 20KG en esta fase inicial.



4. MOCKUP

a. Autenticación



The mockup shows a login interface for 'rackIQ'. The background is a dark blue vertical rectangle with diagonal white lines, flanked by lighter blue areas with similar patterns. At the top center is the 'rackIQ' logo in white and teal. Below it is a circular icon containing a stylized person. The main form is a white rectangle with a thin grey border. It contains two input fields: 'Email' and 'Password', each with a placeholder 'Value'. Below these is a dark blue 'Sign In' button. Under the button is a link that says 'Forgot password?'. At the bottom of the form is a dark blue button with the text 'registrarse' in white.

- b. Sdsd
- c. Sdsd
- d. Sdsds
- e. Ddsd
- f. Sdsd
- g. sdsdsd

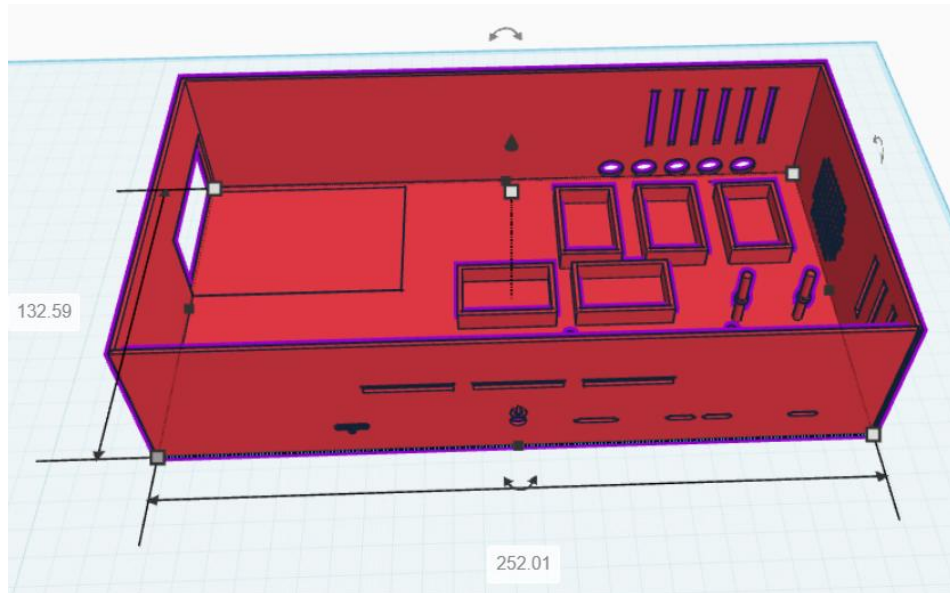
5. cambios y justificaciones

a. cambio en los case

el principal cambio es el disenio del case. Porque en el primer disenio no se tivo en cuenta la incorporacion de una raspberry pi, asi como el consumo energetico de los modulos HX711, si bien es cierto que un ESP32 podria alimentar a los modulos no seria lo mas recomendable, ya que el SoC puede alenctarse, puede genrear ruido en las lineas de lectrua lo que va a afectar las lectruas del HX711, puede reiniciarse. Por lo que se opto por asignar una fuente de poder conmutada dedicada a suministrar de energia a los componentes, lo que significa que el tamano del case deberia se aumentar en tamano para que los compinentes puedan caber perfectamente.



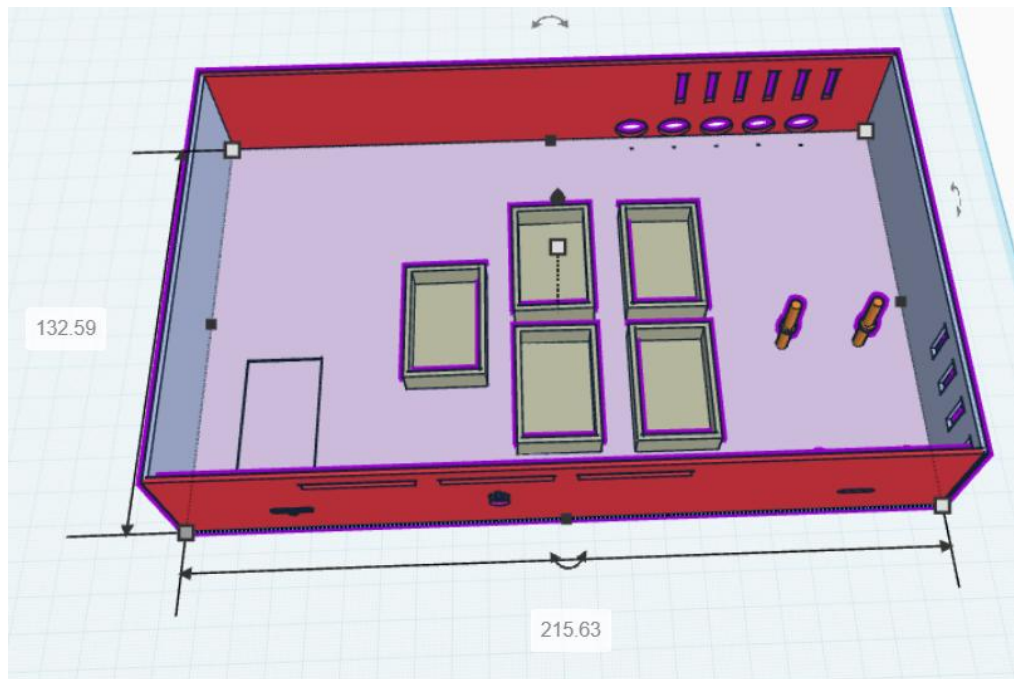
Como se ve en la imagen de la primera muestra solo se tomaba en cuenta el SoC y los modulos y se planeaba que las celdas fueran integradas de lo menos invasivo en el equipo actual que se tuviera, pero la manera mas simple fue realizar un estante aapрте en donde estaraі dedicado al monitoreo.



En esta segunda version se ha tomado en cuenta todos los componentes necesarios para que el sistema funcione en optimas condiciones. Asi como una distancia considerable para que las frecuencias que puedas transmitir la fuente de poder no afectaras a los modulos HX711 y puedan otorgar una lectura lo mas limpia que se pueda, eso tambien tomando en cuenta el tipo de cable a utilizar. Medida 132.59 X 252.01 mm. Tendra la capacidad de integrar un ventilador de 40 x 40 x 10 mm, a 5v, para poder enfriar los componentes en especial la ESP32 que tiende a calentarse con WIFI activado y se necesita el WIFI activado todo el tiempo, por el tipo de producto no podria implementar sleep mode. Por esa razon de opto por una fuente de poder frente a una pila que alimente,

¿Porque una fuente de poder conmutada frente a una batería recargable? Esto, por el tipo de mercado que va orientado el producto, seria bastante incomodo que puede que cada 2 dias, cada 3 dias, dependiendo el tipo de batería tendria que estar conectandose o que se termine la batería en mitad de la noche y hasta el dia siguiente el encargado se daría cuenta. Como se dijo con anterioridad el sistema no puede entrar en sleep mode. Lo mejor seria una fuente de poder conectada a la red eléctrica que entregue la corriente necesario sin interrupciones.

El otro cambio fue el diseño de un nodo secundario, este igual cambio para mantener un diseño muy parecido al nodo principal y este igual necesitara un convertidor de AC-DC por la misma razon. Un ESP no es recomendable que alimente a los modulos HX711. El cual necesitara un espacio para un circuito secundario necesario para la proteccion del circuito.



Es mas chico que sigue teneido espacio necesario para organizar el cableado de los 5 modulos.

- b. sdsd**
- c. sdsd**
- d. sd**

- 6.
- 7. sdsd
- 8. sdds